

Technical Assessment

Record Ingestion & Reporting

1. Ground rules

- Build it in the language and framework you are strongest in.
- Use whatever tools you normally use, including AI assistants. We do not care how the code was produced — we care that you understand it, because the next stage is a short session where you will change this code with us watching.
- This is a small problem on purpose. We would much rather see a modest thing done carefully than a large thing done in a hurry.
- The specification does not answer every question. Some of those gaps are the exercise. Noticing them and deciding what to do is a large part of what we are assessing.

If something is genuinely blocking you, email us. Asking a good question is not a mark against you.

2. The scenario

A system somewhere produces records and drops them into a file. Another team consumes that file. The file is not clean — the same record sometimes appears twice, fields are occasionally missing, dates arrive in whatever format the sending system felt like using, and some values are plainly wrong.

Your job is to take that file, work out which records are usable, store them, and be able to answer questions about them afterwards. The records you throw away matter as much as the ones you keep — somebody will eventually ask why a particular record never showed up, and "it was dropped" is not an acceptable answer.

3. The records

Each record looks roughly like this:

```
{ "id": "r-0001",  
  "source": "alpha",  
  "recordedAt": "2026-03-14T09:12:00Z",  
  "value": 42,  
  "status": "OK" }
```

The rules: id is unique, source is a short string, recordedAt is a point in time, value is an integer between 0 and 100, and status is one of OK, WARN or FAIL.

Ship a sample file of at least 200 records with your code. It must contain, at minimum:

- The same id appearing more than once — sometimes identical, sometimes with a different value
- Records with a missing field
- Dates in at least two different formats
- Values outside the allowed range
- A status that is not on the list
- Empty or whitespace-only strings where text is expected

4. Requirements

R1 Read the file, check every record against the rules above, and store the ones that pass.

R2 Every record you reject must be recoverable afterwards, together with the reason it was rejected. Nothing disappears silently.

R3 Running your program twice over the same file must not duplicate anything. Think about what "the same record" means before you decide how.

R4 Provide a way to query what you stored — filter by source, by status, and by a date range. A small HTTP API or a command-line query, your choice.

R5 At the end of a run, report how many records were accepted and how many were rejected, with the rejections grouped by reason.

R6 Write an ASSUMPTIONS.md listing every decision you had to make that the specification did not answer for you, and why you decided that way. This file matters as much to us as the code.

5. Constraints

- Any language and framework you like. Any storage you like, including a plain file or an embedded database — but be ready to explain the choice.
- It must run from a clean checkout with instructions that fit on half a page.
- We have not told you what tests to write, or how many. That choice is part of the exercise.

6. What the specification leaves out

There are several questions above that we have not answered. That is deliberate. In most cases more than one answer is defensible, and we care far more about your reasoning than about which one you pick.

Here is one, so the intent is clear. The same id appears twice with two different values. Do you keep the first one you saw, the last one, the one with the newer timestamp, or neither — and do you treat that as a rejection, or as an update? Each answer has consequences for whoever reads your data later. Decide, implement it, and explain the trade-off in a short paragraph.

There are more like it. Finding them is the exercise.

7. What to send

- A Git repository, with history intact — we would rather see your commits than one squashed drop
- README.md: how to run it, how to query it, the design you chose and why, and anything you would do differently with more time
- ASSUMPTIONS.md, as described in R6
- Your sample file and your tests

A small, correct, well-explained submission beats an ambitious broken one. We read the README and ASSUMPTIONS.md before we read the code.